

Ir Além 2 | Automação e dados

Base acadêmica e arquitetura

AIRPA, capítulo 2, apresenta SQLite (pp. 9-11), Isolation Forest (pp. 23-30) e MongoDB com Docker/pymongo (pp. 46-48). A solução usa essas tecnologias com Python periódico. SQLite é embutido no container do robô; MongoDB é um serviço do Compose com rede interna e volume próprio. A interpretação textual reutiliza Gemini, conforme o capítulo PCV.

Preparação dos dados

A carga determinística usa `random_state`/semente 42 e um paciente fictício SIM-001. São 240 medições de treino e 63 de monitoramento, separadas por campo `dataset`. Os atributos são pressão sistólica, diastólica, frequência cardíaca e adesão. Existem três casos controlados: 301 com desvio injetado, 302 de referência e 303 com dado ausente. As distribuições escolhidas são artificiais, não faixas clínicas.

Ciclo do robô

O robô cria um identificador de execução, registra o início, prepara a carga idempotente e treina o modelo com o subconjunto de treino. Busca medições ainda não avaliadas para a versão do modelo, valida os atributos, calcula predição/escore e persiste avaliações e alertas. Replica os eventos ao MongoDB, interpreta mensagens pendentes quando há chave/cota e registra o término. O intervalo padrão de 60 segundos é configurável.

Técnica de IA

`IsolationForest` usa 100 árvores, `contamination=0.08` e `random_state=42`. O parâmetro 0.08 calibra o corte no treino; não representa prevalência clínica nem garante 8% no monitoramento. Predição -1 gera `anomalia_estatistica`; demais casos recebem `sem_desvio_detectado`. Valores inválidos recebem `dado_invalido` e não entram no modelo. A versão é o hash de parâmetros, atributos, conjunto de treino e versão da implementação.

Persistência e rastreabilidade

SQLite mantém patients, measurements, evaluations, alerts, runs e outbox. Avaliações usam chave composta de medição e modelo; alertas usam identificador estável. MongoDB mantém runs, events e messages. Cada evento se vincula à medição/mensagem e execução de origem, com versão/modelo e data. A interpretação de mensagem conserva texto original e fatos com evidências.

Recuperação de falhas

Resultados e eventos são gravados em uma transação local. A outbox é enviada ao MongoDB por upsert e marcada entregue somente após sucesso. Uma falha depois do envio pode causar reenvio, mas o mesmo identificador evita duplicação. Não há transação distribuída: os bancos convergem por reconciliação. A falha de conexão conserva eventos pendentes e status observável. Um bloqueio de arquivo no volume impede sobreposição de robôs.

Resultados observados

Em execução real com SQLite e MongoDB nos containers, o primeiro ciclo processou 63 medições, gerou 11 alertas estatísticos e deixou zero eventos pendentes de sincronização. Cinco testes do robô aprovaram os casos controlados, idempotência e recuperação da fila após falha simulada. As três mensagens do MongoDB foram interpretadas pelo Gemini em chamadas reais, com evidências e vínculo às execuções. A recriação dos containers conservou os dados e os ciclos seguintes não duplicaram os alertas.

Operação e limites

`docker compose --profile automacao up --build -d` inicia o fluxo. A aba Monitoramento mostra quantidades e ciclos; `docker compose logs robot` mostra identificadores e status. O comando `--once` permite execução isolada após parar o robô periódico. O conjunto inicial é estático: próximos ciclos não criam medições automaticamente; novas leituras podem ser inseridas por `automation/add_measurement.py`. Os dados persistem nos volumes durante encerramento normal.

Referências e entregáveis

AIRPA: 2TIAOA - Fase 5 - Cap02 - Do Banco de Dados à Automação Inteligente, pp. 9-11, 23-30 e 46-48. Código: `automation/robot.py`, `automation/data.py` e `automation/analysis.py`. Esquema SQL: `database/relational/schema.sql`. Estruturas documentais: `database/nonrelational/README.md`. Execução, testes e limitações: `README.md` e `docs/VALIDACAO.md`.